

# Denoising Diffusion Probabilistic Models (DDPM)

*Jonathan Ho, Ajay Jain, Pieter Abbeel*

汇报人：李欣峰

# 研究背景

## 生成式深度学习与扩散概率模型的兴起

深度生成模型（Deep Generative Models）在图像合成、语音生成和多模态交互等诸多数据领域展现出了极高的应用价值。然而，传统的几大主流生成范式在实际应用中各自存在明显的局限性：

**GANs：**虽然能够生成高保真度、高锐度的图像，但其训练过程极不稳定，容易出现模式崩溃且难以进行精确的似然值评估与表达。

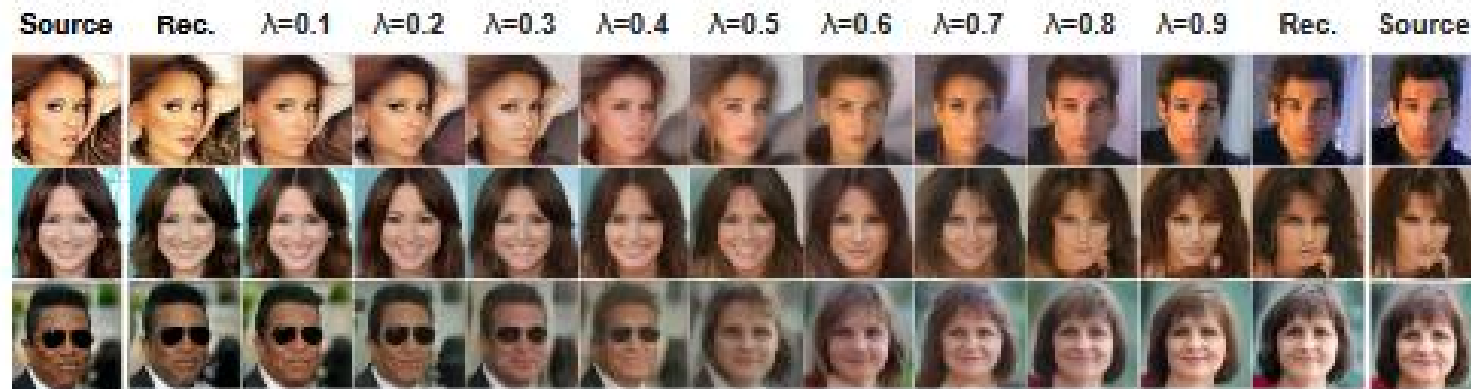
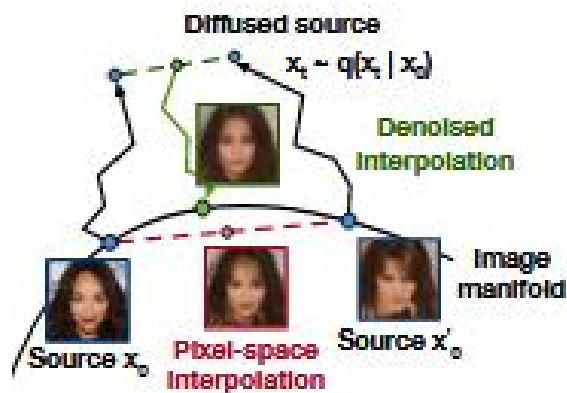
**自回归模型 (Autoregressive Models)：**虽然具有可处理显式似然估计的优势，但由于其样本生成需要按像素或序列逐个进行，导致采样计算速度极慢，在高分辨率数据上难以扩展。

**VAEs与流模型：**VAEs 易导致生成的图像模糊，而流模型则对网络架构具有严格的数学限制（如必须可逆且雅可比矩阵易算），这在很大程度上限制了模型的表达能力。

**基于能量的模型 (EBMs) 与得分匹配 (Score Matching)：**在进行高维数据的精确采样和似然估计时，仍面临计算成本高昂和样本质量不稳定的挑战。

## DDPM 的突破

论文从深度生成模型在“训练稳定性、采样效率与样本质量”三者之间难以兼顾的长期历史痛点。通过对扩散概率模型的数学重构与简化，本论文不仅打破了 GANs 在图像生成领域的垄断地位，也为后续席卷全球的扩散模型研究浪潮（如之后一脉相承的 Stable Diffusion, Sora 等技术）奠定了坚实的基石

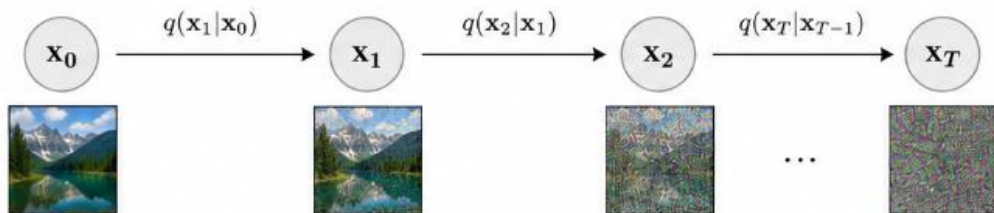


## **Related Work**

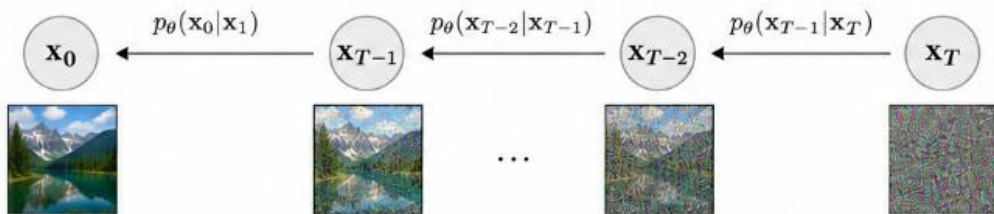
## 结构对比：DDPM和VAE

### DDPM

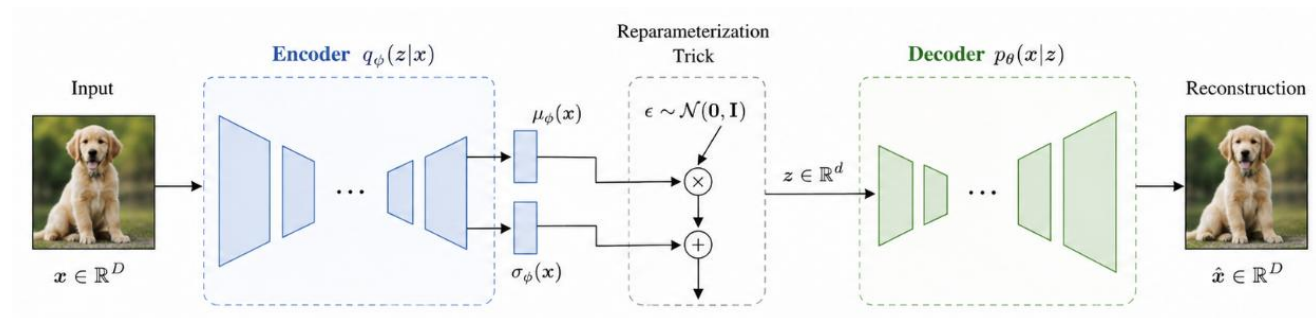
#### 1. Forward process (add noise)



#### 2. Reverse process (remove noise)



### VAE



都是隐变量生成模型体系，我理解DDPM算是一种VAE的变体

## 功能演进：DDPM vs 自回归模型

传统的自回归模型按空间像素顺序生成（慢，受限）。  
DDPM 可被视为一种具有广义位序的自回归模型，实现从宏观“概念”到微观的渐进式解码

# 研究方法



# DDPM的整体流程

前向过程

Reverse过程

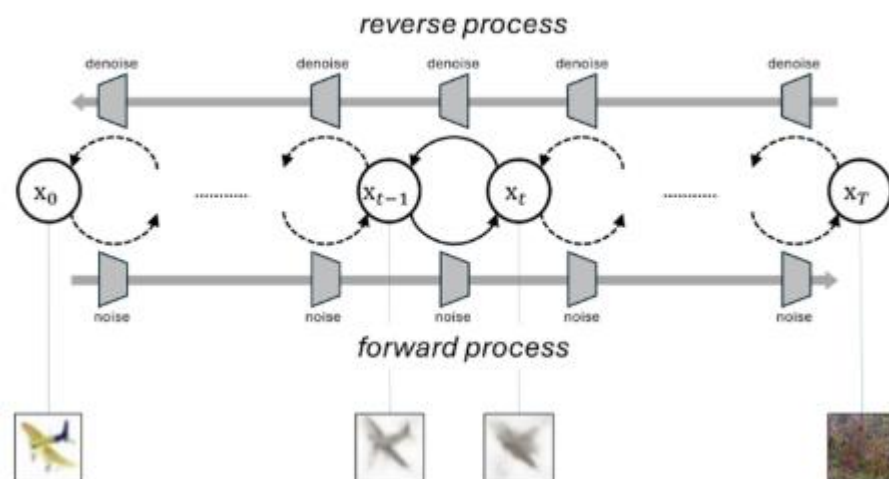
## Algorithm 1 Training

```
1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
      $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2$ 
6: until converged
```

## Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

通过重参数化之后，训练过程不用一步一步计算，可以通过公式直接得到t步的 $\mathbf{x}_t$ ，从而减少了大量计算



# 损失函数

## 训练目标L定义

$$\mathbb{E}[-\log p_{\theta}(\mathbf{x}_0)] \leq \mathbb{E}_q \left[ -\log \frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] = \mathbb{E}_q \left[ -\log p(\mathbf{x}_T) - \sum_{t \geq 1} \log \frac{p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})} \right] =: L$$

我们通过优化负对数似然，通过优化它的一个上界（通过jensen不等式得到），  
因为如果直接对公式左边的真正目标进行优化，需要积分掉所有中间变量（下式）

$$p_{\theta}(x_0) = \int p_{\theta}(x_{0:T}) dx_{1:T}$$

所以引入最小上界作为训练目标L

$$\mathbb{E}_q \left[ -\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} \right]$$

## 优化目标分子部分

$$p_{\theta}(\mathbf{x}_{0:T}) := p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)$$

这是模型要学习的reverse过程


$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \boldsymbol{\Sigma}_{\theta}(\mathbf{x}_t, t))$$

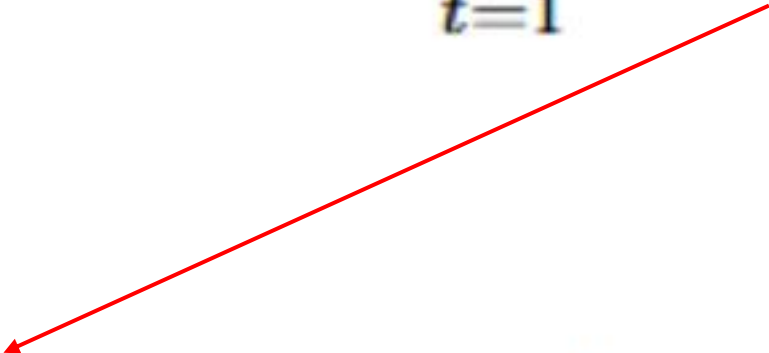
这一步就是模型在reverse过程中一步一步去噪的过程

我理解为模型认为这条完整路径的概率

## 优化目标分母部分

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) := \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}),$$

这一步则是训练过程中向前加噪的过程，从真实图片 $\mathbf{x}_0$ 逐步加噪


$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

其中这个过程 $\beta$ 通常是人为规定的，所以是一个固定的马尔可夫过程

我理解为前向加噪过程中产生这条路径的概率

$$\mathbb{E}_q \left[ -\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right]$$

上下拆分之后我理解为：我们需要尽可能去让模型逆向去噪的路径选择（待学习）和加噪的路径（已知）重合度更高，重合度越高这个目标值会越小

$$D_{KL}(q(\text{前向真实轨迹}) \parallel p_\theta(\text{逆向模型轨迹}))$$

如果上下两个分布差异很大的话，KL散度无穷大，损失函数就会爆炸

$$L = \mathbb{E}_q \left[ -\log p(\mathbf{x}_T) - \sum_{t=1}^T \log \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})} \right]$$

但是，现在的目标函数存在问题：两个分布是一前一后方向问题，reverse的每一步都会有很多可能，所以会导致方差极大，所以我们需要将其改写

## 目标函数的变换

$$L = \mathbb{E}_q \left[ -\log p(\mathbf{x}_T) - \sum_{t=1}^T \log \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})} \right] \xrightarrow{\text{贝叶斯公式展开}} q(\mathbf{x}_t|\mathbf{x}_{t-1}) = q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) = \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \cdot q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}$$

比如T=3:

$$\text{Inside} = -\log p(\mathbf{x}_3) - \log \frac{p_\theta(\mathbf{x}_2|\mathbf{x}_3)}{q(\mathbf{x}_3|\mathbf{x}_2)} - \log \frac{p_\theta(\mathbf{x}_1|\mathbf{x}_2)}{q(\mathbf{x}_2|\mathbf{x}_1)} - \log \frac{p_\theta(\mathbf{x}_0|\mathbf{x}_1)}{q(\mathbf{x}_1|\mathbf{x}_0)}$$

分母是向前的，  
无法和逆向的分  
子凑成散度

相邻项之间抵消  
后，只看分母，  
分母只剩下

$$\sum_{t=1}^3 \log q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \log q(\mathbf{x}_1|\mathbf{x}_2, \mathbf{x}_0) + \log q(\mathbf{x}_2|\mathbf{x}_3, \mathbf{x}_0) + \log q(\mathbf{x}_3|\mathbf{x}_0)$$

同理，推广到任  
意步数T

$$\sum_{t=1}^T \log q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \sum_{t=2}^T \log q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) + \log q(\mathbf{x}_T|\mathbf{x}_0)$$

## 将分母与分子项合并

先合并边界项

$$+ \log q(\mathbf{x}_3|\mathbf{x}_0) - \log p(\mathbf{x}_3) = \log \frac{q(\mathbf{x}_3|\mathbf{x}_0)}{p(\mathbf{x}_3)}$$



$$D_{KL}(q(\mathbf{x}_3|\mathbf{x}_0) \parallel p(\mathbf{x}_3))$$

即LT项 (纯噪声项)

合并中间项  
t>1

$$\begin{aligned} \log q(\mathbf{x}_2|\mathbf{x}_3, \mathbf{x}_0) - \log p_\theta(\mathbf{x}_2|\mathbf{x}_3) &= \log \frac{q(\mathbf{x}_2|\mathbf{x}_3, \mathbf{x}_0)}{p_\theta(\mathbf{x}_2|\mathbf{x}_3)} \\ \log q(\mathbf{x}_1|\mathbf{x}_2, \mathbf{x}_0) - \log p_\theta(\mathbf{x}_1|\mathbf{x}_2) &= \log \frac{q(\mathbf{x}_1|\mathbf{x}_2, \mathbf{x}_0)}{p_\theta(\mathbf{x}_1|\mathbf{x}_2)} \end{aligned}$$



$$\sum_{t=2}^3 D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))$$

中间去噪项，上一轮提到的两条路径要尽量相似，即L(T-1)项

$$- \log p_\theta(\mathbf{x}_0|\mathbf{x}_1)$$

→ 最后一小步的原图像，即L0

推广到任意步数，目标函数分解为

$$\mathbb{E}_q \left[ \underbrace{D_{KL}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right]$$



L1左右都是高斯分布左右两侧都是高斯分布，他们之间的散度可以用一个公式直接计算

$$D_{KL}(\mathcal{N}_1 \parallel \mathcal{N}_2) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

由于方差都是定值  $\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$  所以得到  $D_{KL} \propto (\mu_1 - \mu_2)^2$

于是，计算两个高斯分布的散度就变成了计算他们均值之间的距离，就是L2损失函数

$$L_{t-1} \propto \|\tilde{\mu}_t(\mathbf{x}_0, \mathbf{x}_t) - \mu_\theta(\mathbf{x}_t, t)\|^2$$

# 损失函数重构 (L:t-1重构)

$$\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t \quad \text{代入KL散度公式}$$

$$L_{t-1} = \mathbb{E}_q \left[ \frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)\|^2 \right] + C$$

真实值 $\mu$ 的计算严重依赖 $x_0$ ，所以通过重参数化将 $x_0$ 替换

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$



$$\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon)$$

最后得到

$$\mathbb{E}_{\mathbf{x}_0, \epsilon} \left[ \frac{1}{2\sigma_t^2} \left\| \frac{1}{\sqrt{\bar{\alpha}_t}} \left( \mathbf{x}_t(\mathbf{x}_0, \epsilon) - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right) - \mu_\theta(\mathbf{x}_t(\mathbf{x}_0, \epsilon), t) \right\|^2 \right]$$

由公式可以看出，左边真正未知参数只有噪声  $\epsilon$ ，所以最终右边目标是为了和左边相等，干脆直接去预测左边的  $\epsilon$

于是重新定义了右边

$$\mu_{\theta}(\mathbf{x}_t, t) := \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right)$$

和左边完全一致

带入后，只剩下

$$\mathbb{E}_{\mathbf{x}_0, \epsilon} \left[ \frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right]$$

$$\left\| \epsilon - \epsilon_{\theta}(\mathbf{x}_t, t) \right\|^2$$

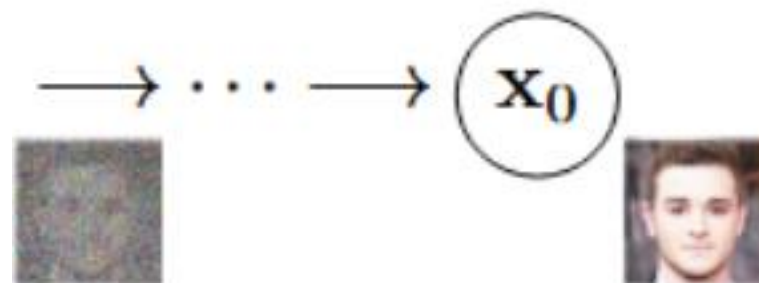
| Objective                                                        | IS                                | FID         |
|------------------------------------------------------------------|-----------------------------------|-------------|
| <b><math>\tilde{\mu}</math> prediction (baseline)</b>            |                                   |             |
| $L$ , learned diagonal $\Sigma$                                  | $7.28 \pm 0.10$                   | 23.69       |
| $L$ , fixed isotropic $\Sigma$                                   | $8.06 \pm 0.09$                   | 13.22       |
| $\ \tilde{\mu} - \tilde{\mu}_{\theta}\ ^2$                       | –                                 | –           |
| <b><math>\epsilon</math> prediction (ours)</b>                   |                                   |             |
| $L$ , learned diagonal $\Sigma$                                  | –                                 | –           |
| $L$ , fixed isotropic $\Sigma$                                   | $7.67 \pm 0.13$                   | 13.51       |
| $\ \tilde{\epsilon} - \epsilon_{\theta}\ ^2 (L_{\text{simple}})$ | <b><math>9.46 \pm 0.11</math></b> | <b>3.17</b> |

前面系数被扔掉，因为实验过程中发现t在很小时，这团系数权重太大，带着系数会将注意放在微小细节上，而忽略宏观轮廓，于是抛弃系数，在消融实验中取得到最佳得效果

# 独立解码器（最后一步T）

Reverse的最后一步，设为一个独立解码器

我们预测出来的结果服从高斯分布，是概率密度函数，而像素点是在0-255范围内，为了得到每个格子最后的值，我们在计算时需要数据缩放，将0-255压缩在[-1, 1]之间



每个像素点会得到一个分布，均值为 $\mu$ ，于是 $\mu$ 会对应压缩前的一个数值，对 $\pm(1/255)$  范围积分得到概率，再算得像素值

$$p_{\theta}(\mathbf{x}_0|\mathbf{x}_1) = \prod_{i=1}^D \int_{\delta_{-}(x_0^i)}^{\delta_{+}(x_0^i)} \mathcal{N}(x; \mu_{\theta}^i(\mathbf{x}_1, 1), \sigma_1^2) dx$$
$$\delta_{+}(x) = \begin{cases} \infty & \text{if } x = 1 \\ x + \frac{1}{255} & \text{if } x < 1 \end{cases} \quad \delta_{-}(x) = \begin{cases} -\infty & \text{if } x = -1 \\ x - \frac{1}{255} & \text{if } x > -1 \end{cases}$$

**experiments**

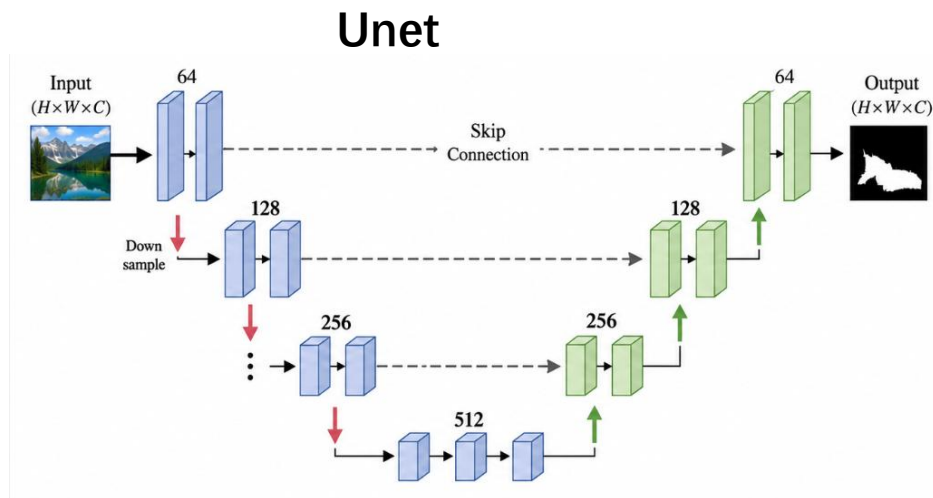
# 实验设置

## 超参数设置

$$T=1000$$

$$\beta_1 = 10^{-4} \xrightarrow{\text{线性增加}} \beta_T = 0.02$$

## backbone



我理解是因为初期图像还很清晰，如果 $\beta$ 太大，会直接损失很多细节，网络可能达不到精细的修复能力，而后期图像已经很糊了，需要快速洗清残存信息

为了让unet分清自己干的任务是哪一步的任务，于是用了time embedding，类似于位置编码，让backbone知道自己处于哪一步

低分辨率位置引入了注意力机制  
应该是为了减少计算量，同时提供全局大局观



---

**Algorithm 3** Sending  $\mathbf{x}_0$ 

---

```
1: Send  $\mathbf{x}_T \sim q(\mathbf{x}_T|\mathbf{x}_0)$  using  $p(\mathbf{x}_T)$ 
2: for  $t = T - 1, \dots, 2, 1$  do
3:   Send  $\mathbf{x}_t \sim q(\mathbf{x}_t|\mathbf{x}_{t+1}, \mathbf{x}_0)$  using  $p_\theta(\mathbf{x}_t|\mathbf{x}_{t+1})$ 
4: end for
5: Send  $\mathbf{x}_0$  using  $p_\theta(\mathbf{x}_0|\mathbf{x}_1)$ 
```

---

把生成图片模型伪装成一个  
对讲机传输模型：图像的发送与接收

发送端

不传原图，通过顺着马尔可夫链把原图加噪成体积小、完全随机的噪声发送，再顺着时间倒退把每一步去掉的误差作为修正信息发送

---

**Algorithm 4** Receiving

---

```
1: Receive  $\mathbf{x}_T$  using  $p(\mathbf{x}_T)$ 
2: for  $t = T - 1, \dots, 1, 0$  do
3:   Receive  $\mathbf{x}_t$  using  $p_\theta(\mathbf{x}_t|\mathbf{x}_{t+1})$ 
4: end for
5: return  $\mathbf{x}_0$ 
```

---

接收端

接收到纯噪声后，又用接收的修正数据加上模型预测的噪声还原x0



Figure 3: LSUN Church samples. FID=7.89



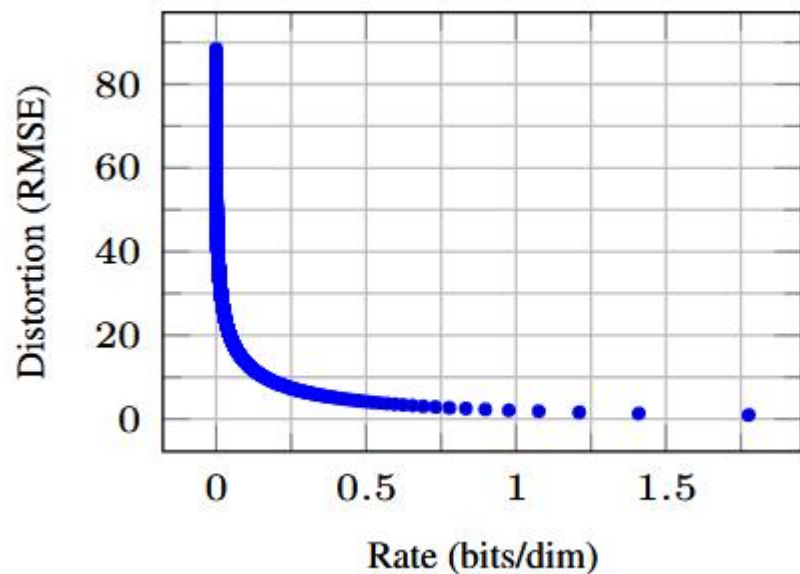
Figure 4: LSUN Bedroom samples. FID=4.90

| 实验组合         | 神经网络预测什么                    | 损失函数用哪个                | 方差 ( $\Sigma$ ) 怎么处理 | 最终实验结果 (FID/稳定性)     |
|--------------|-----------------------------|------------------------|----------------------|----------------------|
| 组合 A (传统基线)  | 图像的长相均值 $\mu$               | 原版变分下界 (带复杂系数)         | 固定为常数                | 表现还可以, 但不是最好         |
| 组合 B         | 图像的长相均值 $\mu$               | 简化 $MSE$ 损失 (扔掉系数)     | 固定为常数                | 效果很差                 |
| 组合 C         | 噪声 $\varepsilon$ 或 均值 $\mu$ | 原版变分下界 (带复杂系数)         | 让网络动态学习              | 训练直接崩溃               |
| 组合 D (DDPM ) | 注入的噪声 $\varepsilon$         | 简化 $L_{simple}$ (扔掉系数) | 固定为常数                | 全场最佳 (FID 3.17)且极其稳定 |

用一个镜像对立公式来探讨失真度与数据的关系，下面是均方损失函数

$$\mathbf{x}_0 \approx \hat{\mathbf{x}}_0 = (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(\mathbf{x}_t)) / \sqrt{\bar{\alpha}_t}$$

$$\sqrt{\|\mathbf{x}_0 - \hat{\mathbf{x}}_0\|^2 / D}$$



这图是结果中对讲机模型接收的数据流和失真度的关系，我们可以看出，在数据量比较多（图像复杂）的情况下，失真度反而很低

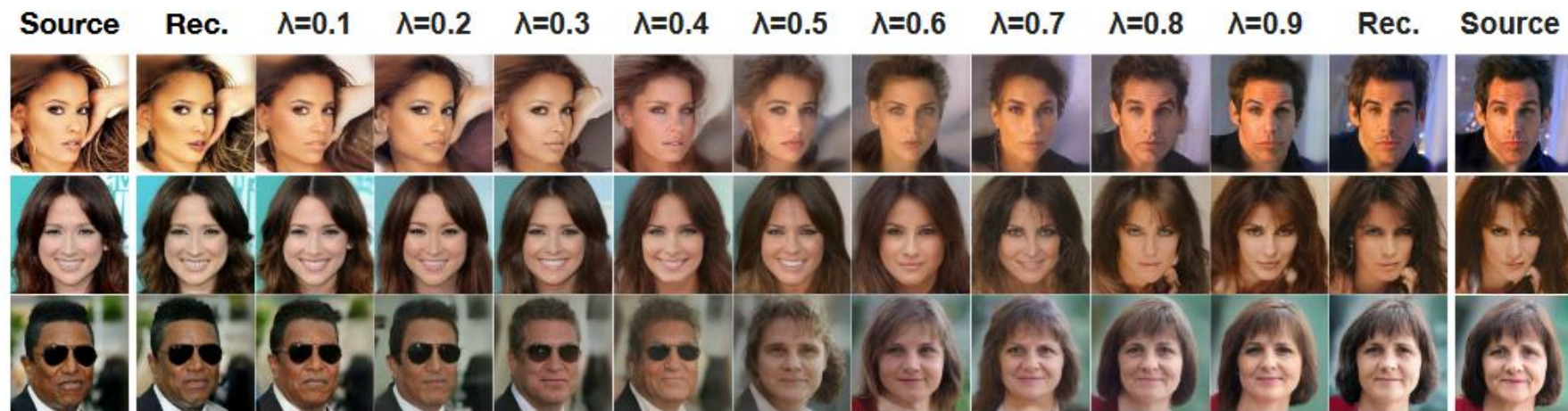
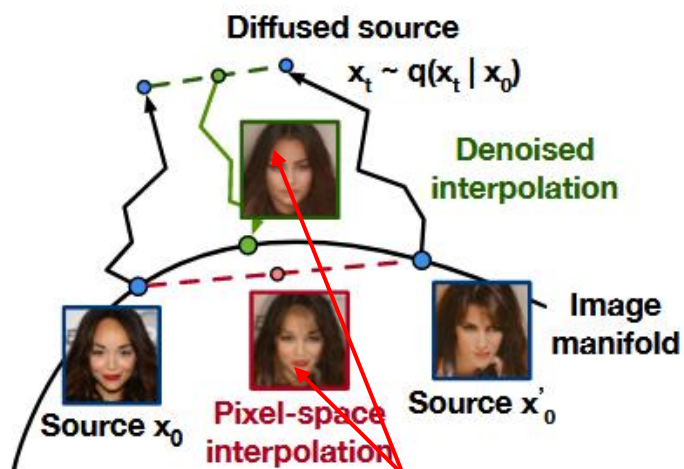
在数据量稍微多一点的情况下，失真度会暴跌，说明传输中最先传输的是一些宏观的语义，也就是最能描述这幅图的像素，然后再来慢慢补充细节



实验结果中可以看到，在中间步骤我们便基本可以看出物体轮廓来，而不是要等到最后，这就是公式15能带来的，因为我们有模型的权重和 $x_t$ 在，所以每一步都能预测出一个 $x_0$ ，公式15理解为一个剧透

$$\mathbf{x}_0 \approx \hat{\mathbf{x}}_0 = (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(\mathbf{x}_t)) / \sqrt{\bar{\alpha}_t}$$





用插值，将左边和右边的人脸根据 $\lambda$ 的值，插值混合得到一张人脸，可以看到，是及其混乱的，特别是到0.5的时刻，两张人脸完全不自然

而左边图中的绿线轨迹则是加噪再做线性混合，然后通过unet来去噪得到效果就很好

# Conclusion

证明了把图片当成一条马尔可夫链一步步破坏和重建，在数学上是百分之百走得通、且有完美解析解的

模型去猜噪声，本质上就是在寻找数据分布的得分（梯度方向）。而生成图片时那种退一步、抖一下的加噪走位，正是物理学中微观粒子在能量场中寻找平衡的运动轨迹

实现了按照画面粗细尺度以及语义能量大小的广义自回归，我觉得可以理解为把带宽主要画在核心语义上